

PROJEKTOVANJE SLOŽENIH DIGITALNIH SISTEMA

Hardverski akcelerator za prepoznavanje šahovskih figura zasnovan na normalizovanoj unakrsnoj korelaciji

Od algoritma do implementiranog sistema:

RT model, ASMD dijagram, pakovanje u IP jezgro i analiza integrisanog sistema

Grupa: y25-g10

Sadržaj

1. Uvod

- 1.1. Predmet i obim dokumenta
- 1.2. Polazna tačka: ESL model sistema
- 1.3. Ciljna platforma i korišćeni alati

2. Opis algoritma

- 2.1. Matematička definicija
- 2.2. Optimizacija 1: kvadrirana korelacija (eliminacija korena)
- 2.3. Optimizacija 2: integralna slika (SAT)
- 2.4. Optimizacija 3: egzaktna celobrojna Q1.31 podela
- 2.5. Bitska analiza širina signala
- 2.6. Promenljiva veličina šablona

3. Uklanjanje petlji

- 3.1. Izgradnja integralne slike
- 3.2. Srednja vrednost šablona
- 3.3. Glavna petlja

4. Interfejs i okruženje

- 4.1. Četiri RT interfejsa
- 4.2. Registarska mapa
- 4.3. Portovi RTL jezgra
- 4.4. Memorijsko okruženje

5. ASMD dijagram

- 5.1. Pregled stanja
- 5.2. Faza A: učitavanje slike i izgradnja SAT
- 5.3. Faza B: šablon i njegova srednja vrednost
- 5.4. Faza C: priprema prozora
- 5.5. Faza D: pipeline-ovana MAC petlja
- 5.6. Faza E: normalizacija i upis rezultata

6. Datapath i controlpath

- 6.1. Registri
- 6.2. Funkcionalne jedinice
- 6.3. Sekvencijalni delilac
- 6.4. Memorije
- 6.5. Blok dijagram

7. Analiza posle sinteze i implementacije

- 7.1. Metod
- 7.2. Utrošeni resursi

- 7.3. Kritična putanja i maksimalna frekvencija
- 7.4. Latencija i propusnost
- 7.5. Funkcionalna verifikacija

8. Pakovanje u IP jezgro

- 8.1. Izbor obrasca: slave sa internim memorijama
- 8.2. Dva AXI interfejsa
- 8.3. Regioni unutar memorijskog interfejsa
- 8.4. Kontrolni i statusni registri

9. Integracija u sistem

- 9.1. Topologija
- 9.2. Adresna mapa
- 9.3. Tok podataka po jednom polju table
- 9.4. Dva pravila koja softver mora poštovati
- 9.5. Zašto DMA, iako nije performansni dobitak

10. Analiza integrisanog sistema

- 10.1. Metod
- 10.2. Utrošeni resursi
- 10.3. Vremenska analiza i radni takt
- 10.4. Propusnost sistema
- 10.5. Put do zatvaranja vremenskih ograničenja
- 10.6. Poređenje sa referentnim rešenjem

11. Zaključak

1. Uvod

1.1. Predmet i obim dokumenta

Projekat realizuje hardverski akcelerator za prepoznavanje šahovskih figura sa fotografije table. Slika table dimenzija 720×720 piksela deli se na 64 polja dimenzija 90×90 piksela; za svako neprazno polje računa se normalizovana unakrsna korelacija (engl. *Normalized Cross-Correlation*, NCC) sa šablonima figura, a iz mape skorova se rekonstruiše pozicija u standardnoj Forsyth-Edwards (FEN) notaciji. Korelacija je, prema profilisanju izvršne specifikacije, uzročnik preko 70 % procesorskog vremena i time prirodan kandidat za akceleraciju u programabilnoj logici.

Obim dokumenta. Dokument prati put od algoritma do implementiranog sistema, u tri celine. Prva (poglavlja 2–6) opisuje **algoritam i RT model**: matematičku osnovu i tri optimizacije, uklanjanje petlji, interfejs, dijagram algoritamske mašine stanja i podelu na datapath i controlpath. Druga (poglavlje 7) daje **analizu samostalnog jezgra** posle sinteze i implementacije. Treća (poglavlja 8–10) opisuje **put do sistema na ploči**: pakovanje jezgra u IP blok sa AXI interfejsom, integraciju sa procesorskim sistemom i DMA kontrolerom, i analizu celog integrisanog dizajna.

Sve navedene brojke su izmerene, a ne procenjene, i mogu se reprodukovati skriptama iz repozitorijuma. Gde se merenje razlikuje od ranije objavljene vrednosti ili od referentnog rešenja, razlika je izričito navedena i obrazložena.

1.2. Polazna tačka: ESL model sistema

Sistem je prethodno modelovan na sistemskom nivou (ESL) u okruženju SystemC/TLM-2.0 i dokumentovan u okviru predmeta *Projektovanje elektronskih uređaja na sistemskom nivou*. Taj model definiše particionisanje hardvera i softvera, adresnu mapu, registarski interfejs akceleratora i vremenski budžet, i u ovom projektu se koristi kao **izvršna specifikacija**: RTL model mora dati bit-identičan rezultat.

Iz ESL modela se preuzimaju:

- bitske širine svih signala u datapath-u (ESL dokumentacija, Tabela 2), realizovane u odeljku 2.5;
- registarska mapa i adresni prostor programabilne logike (odeljak 4.2).

1.3. Ciljna platforma i korišćeni alati

Ciljna platforma je razvojna ploča **Zybo Z7-10** sa uređajem **xc7z010clg400-1** (Zynq-7000). Uređaj raspolaže sa 17.600 LUT, 35.200 flip-flopova, 80 DSP48E1 blokova i 60 blokova RAMB36.

Napomena o oznaci uređaja: dokumentacija prethodne faze projekta navodi xc7z010-clg225-2, ali to je bio podrazumevani uređaj alata Vitis HLS, a ne stvarna ploča — **nijedna Digilent ploča sa Zynq-7010 ne koristi kućište clg225**. Isti je čip i isti kapacitet, ali je brzinska klasa -1 umesto -2, dakle sporija. Sve brojke u ovom dokumentu su merene na clg400-1.

Nominalni takt programabilne logike je 100 MHz (period 10 ns) i samostalno jezgro ga ispunjava (odeljak 7.3). U integrisanom sistemu takt je **90,909 MHz**, jer PLL procesorskog sistema za traženih 95 MHz daje 1000/11 — odstupanje od nominalnih 100 MHz je 9,1 % (odeljak 10.3).

RT model je opisan u jeziku **VHDL-2008** i razvijen ručno iz ASMD dijagrama, bez visokonivosa sinteze. Simulacija je izvršena alatom **XSim**, a sinteza i vremenska analiza alatom **Vivado 2025.2**.

2. Opis algoritma

Akcelerator prima segment slike dimenzija do 90×90 piksela i šablon figure dimenzija do 30×30 piksela, i za **svaku** poziciju prozora (u, v) računa kvadrat normalizovane unakrsne korelacije. Rezultat je mapa skorova dimenzija (img_w - tmp_w + 1) × (img_h - tmp_h + 1), koju softver zatim pretražuje tražeći maksimum.

2.1. Matematička definicija

Koeficijent korelacije γ u tački (u, v), za segment slike f i šablon t , definisan je izrazom:

$$\gamma(u,v) = \frac{\sum_{x,y} [f(x,y) - \bar{f}_{u,v}] \cdot [t(x-u, y-v) - \bar{t}]}{\sqrt{\sum_{x,y} [f(x,y) - \bar{f}_{u,v}]^2 \cdot \sum_{x,y} [t(x-u, y-v) - \bar{t}]^2}}$$

gde je $f(x,y)$ intenzitet piksela segmenta, $\bar{f}_{u,v}$ srednja vrednost piksela onog dela slike koji se trenutno preklapa sa šablonom, t piksel šablona, a $\bar{t}$ srednja vrednost celog šablona. Normalizacija oduzimanjem srednjih vrednosti čini meru neosetljivom na promenu osvetljenja, što je i razlog za izbor NCC-a umesto obične korelacije.

Direktna implementacija ovog izraza je za hardver neprihvatljivo skupa: sadrži kvadratni koren, deljenje po svakoj poziciji prozora i ponovno sumiranje prozora u svakoj tački. Sledeća tri odeljka opisuju tri transformacije kojima se izraz svodi na oblik pogodan za realizaciju u programabilnoj logici, **bez gubitka tačnosti**.

2.2. Optimizacija 1: kvadrirana korelacija (eliminacija korena)

Rezultat korelacije se ne koristi kao brojna vrednost, već isključivo za poređenje sa pragom detekcije: figura je prepoznata ako je $\gamma > 0,75$. Pošto je $\gamma \in [-1, 1]$ i pošto se traži maksimum po apsolutnoj vrednosti, poređenje se ekvivalentno može izvesti nad **kvadratom** korelacije:

$$\gamma^2(u,v) > 0,75^2 = 0,5625$$

Time kvadratni koren potpuno nestaje iz datapath-a, a izraz koji hardver računa postaje:

$$NCC^2(u,v) = \frac{(\sum \text{diff_f} \cdot \text{diff_t})^2}{\sum \text{diff_f}^2 \cdot \sum \text{diff_t}^2}$$

gde su $\text{diff_f} = f(x,y) - \bar{f}$ i $\text{diff_t} = t(x,y) - \bar{t}$. Umesto korena i jednog deljenja po tački, ostaju tri akumulacije i jedno deljenje.

2.3. Optimizacija 2: integralna slika (SAT)

Srednja vrednost prozora $\bar{f}_{u,v}$ zavisi od pozicije prozora, pa bi je naivna implementacija računala iznova za svaku poziciju — to je $\text{tmp_w} \times \text{tmp_h}$ sabiranja po **svakoj** od nekoliko hiljada pozicija. Umesto toga, jednom po segmentu se gradi **integralna slika** (engl. *summed-area table*, SAT), u kojoj svaki element sadrži sumu svih piksela iznad i levo od sebe:

$$\text{SAT}[y+1][x+1] = \text{SAT}[y][x+1] + \text{row_sum}(y, 0..x)$$

Suma proizvoljnog pravougaonog prozora tada se dobija u konstantnom vremenu, sa četiri pristupa memoriji:

$$\text{sum_f}(u,v) = \text{SAT}[v+th][u+tw] - \text{SAT}[v][u+tw] - \text{SAT}[v+th][u] + \text{SAT}[v][u]$$

Složenost pripreme je $O(\text{img_w} \times \text{img_h})$ i plaća se jednom po segmentu, dok se trošak po prozoru svodi sa $O(\text{tmp_w} \times \text{tmp_h})$ na $O(1)$. Za realan slučaj (segment 90×90 , šablon 30×30) to je razlika između 900 i 4 sabiranja po svakoj od 3.721 pozicija.

2.4. Optimizacija 3: egzaktna celobrojna Q1.31 podela

Rezultat po poziciji prozora je razlomak iz opsega $[0, 1]$, koji se prenosi kao 32-bitna neoznačena vrednost u formatu **Q1.31** (vrednost 1,0 odgovara heksadecimalnom $0x80000000$). ESL model je ovo računao u pokretnom zarezu, uz zaštitnu granicu `if (ncc2 > 1.0) ncc2 = 1.0` koja je bila potrebna zbog grešaka zaokruživanja tipa `double`.

RT model umesto toga izvodi **tačnu celobrojnu podelu** u proširenom registru:

$$NCC^2 = (\text{num_sq} \ll 31) / \text{den_prod}$$

gde su `num_sq` i `den_prod` 52-bitne veličine, pa se deljenik računa u 83 bita. Pošto se ne koristi aritmetika u pokretnom zarezu, greška zaokruživanja ne postoji, a Cauchy-Schwarz-ova nejednakost garantuje $\text{num_sq} \leq \text{den_prod}$ egzaktno — zaštitna granica iz ESL modela matematički otpada i nije implementirana. Degenerisani slučaj konstantnog prozora ili konstantnog šablona (nulta varijansa, deljenje nulom) obrađuje se eksplicitno: rezultat je tada 0.

2.5. Bitska analiza širina signala

Širine svih signala u datapath-u realizovane su u paketu `ncc_pkg.vhd`. Tabela 1 daje njihov pregled.

Veličina	Biti	Tip u <code>ncc_pkg.vhd</code>
piksel slike / šablona	8	<code>pixel_t</code> — unsigned(7 downto 0)
dimenzije <code>img_w/h</code> , <code>tmp_w/h</code>	8	<code>dim_t</code> — unsigned(7 downto 0)
$\bar{f}$, <code>template_mean</code>	8	<code>mean_t</code> — unsigned(7 downto 0)
<code>diff_f</code> , <code>diff_t</code>	9	<code>diff_t</code> — signed (8 downto 0)
<code>sum_num</code>	27	<code>numacc_t</code> — signed (26 downto 0)
<code>sum_den_f</code> , <code>sum_den_t</code>	26	<code>denacc_t</code> — unsigned(25 downto 0)
<code>num_sq</code> , <code>den_prod</code>	52	<code>sq52_t</code> — unsigned(51 downto 0)
rezultat (Q1.31)	32	<code>result_t</code> — unsigned(31 downto 0)
akumulator integralne slike	32	<code>sat_t</code> — unsigned(31 downto 0)

Tabela 1. Bitske širine signala datapath-a i njihova realizacija u VHDL-u

Označenost razlika. Veličine `diff_f`, `diff_t` i `sum_num` su **označene**: razlika piksela i srednje vrednosti je negativna kad god je piksel tamniji od proseka, a suma proizvoda tih razlika menja znak. Broj bita (9 i

27) određen je opsegom vrednosti, a označenost predznakom.

Zašto je datapath bit-egzaktan. Srednje vrednosti $\bar{f}$ i $\bar{t}$ se zaokružuju na ceo broj **pre** oduzimanja (deljenje sa zaokruživanjem: $(\text{suma} + \text{count}/2) / \text{count}$). Time su `diff_f` i `diff_t` egzaktni celi brojevi, pa se kroz desetine hiljada sabiranja u akumulatorima ne akumulira nikakva greška zaokruživanja. Ovo je ključna projektna odluka: ceo datapath je celobrojan, a jedina veličina u formatu sa fiksnim zarezom je konačni rezultat.

2.6. Promenljiva veličina šablona

Dimenzije `tmp_w` i `tmp_h` su parametri koji se zadaju u vreme izvršavanja, preko komandnih registara, a ne konstante u vreme prevođenja. To je neophodno jer se šabloni figura razlikuju: top je 25×15 , dok su kraljica i kralj do 30×30 . Konstante `MAX_IMG_W` = `MAX_IMG_H` = 90 i `MAX_TMP_W` = `MAX_TMP_H` = 30 predstavljaju samo gornju granicu za dimenzionisanje memorija, a ne pretpostavku o fiksnoj veličini.

Ista fleksibilnost omogućava i grubi prolaz *coarse-to-fine* algoritma na sistemskom nivou (segment 45×45 , šablon 15×15), pri čemu samo jezgro ne mora ništa da zna o toj strategiji — ono obrađuje dimenzije koje mu se zadaju.

3. Uklanjanje petlji

ASMD dijagram ne poznaje konstrukte `for` i `while`: svaka petlja mora biti prevedena u oblik `if-goto`, u kome brojači petlji postaju registri, a telo petlje jedno ili više stanja. U algoritmu postoje tri mesta sa petljama i ukupno četiri brojača.

3.1. Izgradnja integralne slike

Dva brojača, `y` i `x`:

```
y = 0
L1: if (y >= img_h) goto L1_END
    row_sum = 0
    x = 0
    L2: if (x >= img_w) goto L2_END
        row_sum = row_sum + image[y*img_w + x]
        SAT[(y+1)*(img_w+1) + (x+1)] = SAT[y*(img_w+1) + (x+1)] + row_sum
        x = x + 1
        goto L2
    L2_END:
        y = y + 1
        goto L1
L1_END:
```

3.2. Srednja vrednost šablona

Jedan brojač, `p`, kao linearni indeks kroz šablon:

```
p = 0; sum_t = 0
L3: if (p >= tmp_w*tmp_h) goto L3_END
    sum_t = sum_t + templ[p]
    p = p + 1
    goto L3
L3_END:
    template_mean = (sum_t + count/2) / count
```

3.3. Glavna petlja

Četiri ugnježdene nivoa: spoljni par (`v`, `u`) prolazi kroz pozicije prozora, unutrašnji par (`y`, `x`) kroz piksele šablona. Brojači `y` i `x` su isti registri kao u odeljku 3.1 — dve faze nisu istovremeno aktivne, pa se resurs deli.

```

    v = 0
L4: if (v >= res_h) goto L4_END
    u = 0
L5: if (u >= res_w) goto L5_END
    sum_f = SAT[(v+th)*(iw+1)+(u+tw)] - SAT[v*(iw+1)+(u+tw)]
        - SAT[(v+th)*(iw+1)+u]      + SAT[v*(iw+1)+u]
    f_bar = (sum_f + count/2) / count
    sum_num = 0; sum_den_f = 0; sum_den_t = 0
    y = 0
L6: if (y >= tmp_h) goto L6_END
    x = 0
L7: if (x >= tmp_w) goto L7_END
    diff_f = image[(v+y)*img_w + (u+x)] - f_bar
    diff_t = templ[y*tmp_w + x]         - template_mean
    sum_num  = sum_num  + diff_f*diff_t
    sum_den_f = sum_den_f + diff_f*diff_f
    sum_den_t = sum_den_t + diff_t*diff_t
    x = x + 1
    goto L7
L7_END:
    y = y + 1
    goto L6
L6_END:
    if (sum_den_f == 0 or sum_den_t == 0)
        result = 0
    else
        result = (sum_num*sum_num << 31) / (sum_den_f*sum_den_t)
    result_map[v*res_w + u] = result
    u = u + 1
    goto L5
L5_END:
    v = v + 1
    goto L4
L4_END:

```

4. Interfejs i okruženje

4.1. Četiri RT interfejsa

RT metodologija zahteva da se za svaki modul definišu četiri grupe signala: komandni, statusni, ulazni podaci i izlazni podaci. Tabela 2 daje njihovo konkretno značenje kod NCC jezgra.

Interfejs	Realizacija
Komandni	Konfiguracioni registri REG_IMG_W/H, REG_TMP_W/H, REG_IMG_ADDR, REG_TMP_ADDR (upisuju se pre pokretanja) i REG_CTRL (upis jedinice pokreće obradu). Na nivou RTL jezgra to su ulazi img_w/h, tmp_w/h i signal start.
Statusni	Registar REG_STATUS (0 = zauzet, 1 = završeno); na nivou jezgra izlazi busy i done.
Ulazni podaci	Ne prenose se preko porta kao vrednosti, već se čitaju iz memorije preko adresno-podatkovnog para: img_addr_o / img_data_i i templ_addr_o / templ_data_i.
Izlazni podaci	Mapa rezultata se upisuje u memoriju preko result_addr_o, result_data_o i result_wr_o; procesor je čita sa offseta ADDR_RESULTS.

Tabela 2. Realizacija četiri RT interfejsa

Širina izlaznog porta izvedena je funkcionalno iz širina ulaza: pošto su num_sq i den_prod 52-bitni, a količnik posle skaliranja sa 2^{31} po Cauchy-Schwarz-u nikad ne prelazi 2^{31} , rezultat staje u 32 bita bez zasićenja.

4.2. Registarska mapa

Offset	Registar	Opis	Pristup
0x00	REG_IMG_W	širina segmenta slike	U
0x04	REG_IMG_H	visina segmenta slike	U
0x08	REG_TMP_W	širina šablona	U
0x0C	REG_TMP_H	visina šablona	U
0x10	REG_IMG_ADDR	adresa segmenta u deljenoj memoriji	U
0x14	REG_TMP_ADDR	adresa šablona u deljenoj memoriji	U
0x30	REG_CTRL	upis vrednosti 1 pokreće obradu	U
0x34	REG_STATUS	0 = zauzet, 1 = završeno	Č
0x40	ADDR_RESULTS	početak mape rezultata (32 bita po poziciji)	Č

Tabela 3. Registarska mapa NCC jezgra (preuzeta iz ESL modela)

Adresni prostor programabilne logike, definisan još u ESL modelu, je: deljena memorija na 0x4000_0000, prvo NCC jezgro na 0x5000_0000, drugo na 0x5100_0000 i DMA kontroler na 0x6000_0000. Mapa je od početka projektovana tako da odgovara rasporedu adresa opšte namene na Zynq platformi, pa se može preuzeti neizmenjena pri integraciji u blok dizajn.

 **Tabela iznad opisuje izvršnu specifikaciju, ne konačnu realizaciju.** Tri bazne adrese jezgara i DMA kontrolera prenesene su **neizmenjene** (odjeljak 9.2), ali dve stvari se razlikuju, jer je jezgro spakovano kao slave sa internim memorijama a ne kao master:

- deljene memorije na 0x4000_0000 u realizaciji **nema**, pa registri REG_IMG_ADDR i REG_TMP_ADDR ostaju **rezervisani** — nema adrese koju bi procesor javljao;
- REG_STATUS u realizaciji ima **dva odvojena bita** (završeno, zauzeto) umesto jedne vrednosti, jer se stanja *još nije počelo* i *u toku* inače ne mogu razlikovati.

Stvarni registarski interfejs, sa obrazloženjem oba odstupanja, dat je u odeljku 8.4; puna adresna mapa sistema u odeljku 9.2.

4.3. Portovi RTL jezgra

Entitet `ncc_core` ima interfejs prikazan u Tabeli 4. Adresno-podatkovni parovi ponašaju se kao sinhroni memorijski port: adresa se postavlja u jednom taktu, a podatak je validan u narednom.

Port	Smer	Tip	Značenje
clk, rst	in	std_logic	takt i asinhroni reset
start	in	std_logic	pokretanje obrade (jedan takt)
busy, done	out	std_logic	status obrade
img_w, img_h	in	dim_t	dimenzije segmenta
tmp_w, tmp_h	in	dim_t	dimenzije šablona
img_addr_o / img_data_i	out / in	integer / pixel_t	čitanje segmenta
templ_addr_o / templ_data_i	out / in	integer / pixel_t	čitanje šablona
result_addr_o, result_data_o, result_wr_o	out	integer / result_t / std_logic	upis mape rezultata

Tabela 4. Portovi entiteta `ncc_core`

4.4. Memorijsko okruženje

Jezgro ne poseduje memorije iz kojih čita: adresu izdaje, a podatak dobija u narednom taktu, kao kod svakog sinhronog memorijskog porta. Ono što se nalazi *sa druge strane* tog porta je pitanje okruženja, a ne jezgra — i upravo se to okruženje razlikuje između izvršne specifikacije i realizacije:

- **izvršna specifikacija** pretpostavlja *deljenu* memoriju programabilne logike, kojoj pristupa više mastera: procesor, DMA kontroler i oba jezgra;
- **realizacija** koristi obrazac *slave sa internim memorijama*: memorije su privatne za blok, dvoportne, i procesor ili DMA ih pune preko AXI interfejsa dok jezgro stoji. Deljene memorije u sistemu **nema**.

Za sam RT model to ne menja ništa — portovi jezgra su u oba slučaja identični, i zato je jezgro ušlo u IP blok nepromenjeno. Obrazloženje odluke i njene posledice na registarsku mapu su u odeljku 8.1.

Integralna slika je u oba slučaja izuzetak: ona je interna i privatna za jezgro, jer se u fazi izgradnje čita i piše u svakom taktu i ne bi tolerisala deljeni pristup.

5. ASMD dijagram

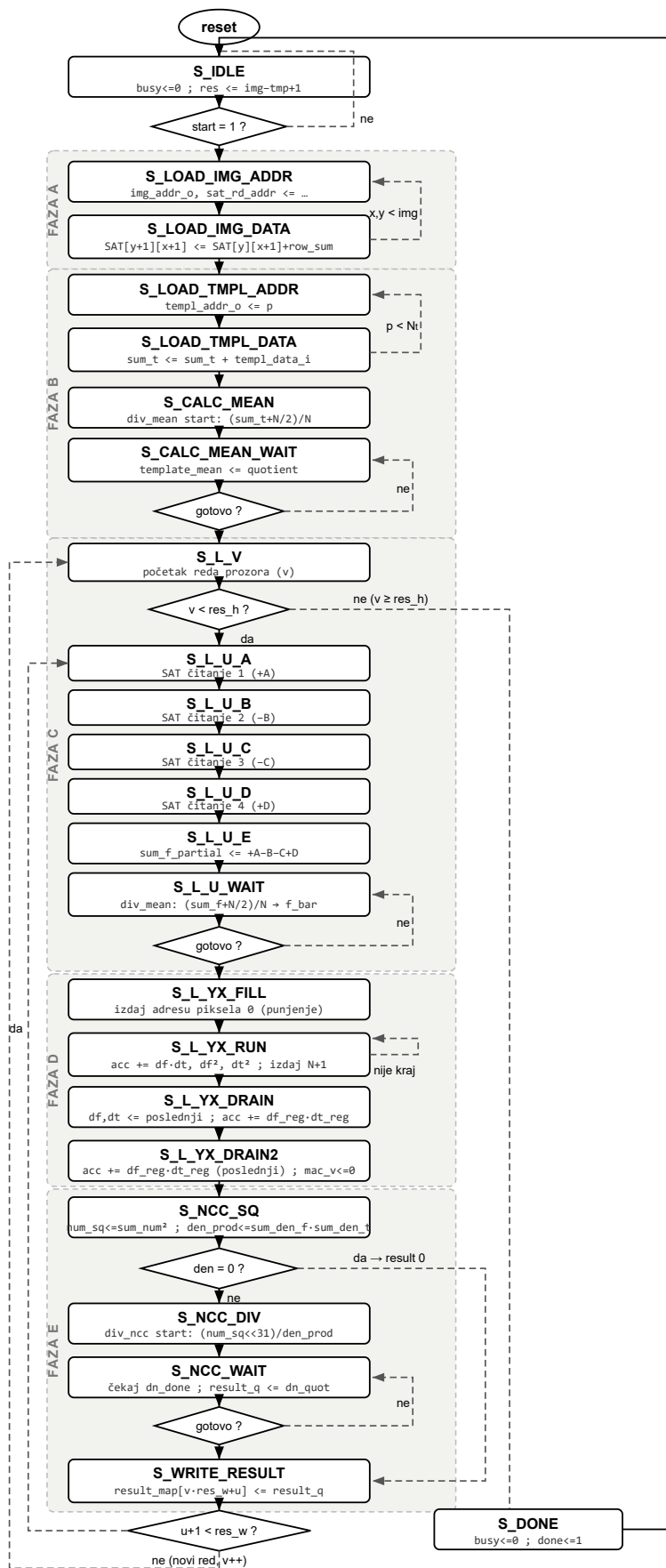
5.1. Pregled stanja

Kontrolna jedinica je Murova mašina stanja sa **23 stanja**, opisana u dvoprocesnom stilu: jedan proces sadrži isključivo registre (`signal_reg <= signal_next`), drugi svu kombinacionu logiku, sa podrazumevanim dodelama na početku i grananjem po `case state_reg`. Memorija integralne slike izdvojena je u treći, poseban proces. Stanja se prirodno grupišu u pet faza, a sva su — svih 23 — nabrojana u Tabeli 5 i prikazana na ASMD dijagramu (Slika 1).

Faza	Stanja	Broj taktova	Namena
—	S_IDLE	1	čekanje na <code>start</code> , preuzimanje dimenzija, računanje <code>res_w/res_h</code>
A	S_LOAD_IMG_ADDR, S_LOAD_IMG_DATA	$2 \cdot \text{img_w}$ $\cdot \text{img_h}$	čitanje segmenta piksel po piksel i istovremena izgradnja integralne slike
B	S_LOAD_TMPL_ADDR, S_LOAD_TMPL_DATA, S_CALC_MEAN, S_CALC_MEAN_WAIT	$2 \cdot \text{tmp_w}$ $\cdot \text{tmp_h}$ $+ \sim 20$	čitanje šablona, sumiranje i deljenje radi dobijanja <code>template_mean</code>
C	S_L_V, S_L_U_A, S_L_U_B, S_L_U_C, S_L_U_D, S_L_U_E, S_L_U_WAIT	$6 + \sim 19$ po prozoru	četiri sekvencijalna čitanja SAT, računanje <code>sum_f</code> i deljenje radi <code>f_bar</code> ; nuliranje akumulatora
D	S_L_YX_FILL, S_L_YX_RUN, S_L_YX_DRAIN	$\text{tmp_w} \cdot$ $\text{tmp_h} + 1$ po prozoru	pipeline-ovana akumulacija tri sume nad pikselima šablona
D	S_L_YX_DRAIN2	1	Akumulira poslednji registrovani par u sve tri sume (<code>sum_num</code> , <code>sum_den_f</code> , <code>sum_den_t</code>); poništava <code>mac_v</code> . Dodato u Koraku 8b uz MAC protočnost.
E	S_NCC_SQ	1	Provera <code>den = 0</code> ; ako nije, registruje <code>num_sq <= sum_num²</code> i <code>den_prod <= sum_den_f · sum_den_t</code> . Dodato u Koraku 8b da kvadriranje ne ulazi kombinaciono u 83-bitni delilac.
E	S_NCC_DIV, S_NCC_WAIT, S_WRITE_RESULT	~ 84 po prozoru	Startuje <code>div_ncc</code> nad registrovanim vrednostima: (<code>num_sq << 31</code>) / <code>den_prod</code> ; upis rezultata.
—	S_DONE	1	spuštanje <code>busy</code> , postavljanje <code>done</code>

Tabela 5. Svih 23 stanja kontrolne jedinice, grupisano po fazama

Slika 1 prikazuje ASMD dijagram. Pravougaonici su stanja sa pripadajućim RT operacijama, rombovi su uslovni blokovi (grananja), a isprekidane linije povratne grane petlji.



Slika 1. ASMD dijagram kontrolne jedinice NCC jezgra (svih 23 stanja, pet faza)

5.2. Faza A: učitavanje slike i izgradnja SAT

Segment se čita piksel po piksel, a integralna slika se gradi **u istom prolazu** — nije potrebna zasebna faza. Par stanja postoji zbog kašnjenja sinhronne memorije: u stanju `S_LOAD_IMG_ADDR` izdaju se adresa piksela i adresa elementa integralne slike iznad tekućeg, a u stanju `S_LOAD_IMG_DATA` oba podatka su validna, pa se računa nova vrednost `row_sum` i upisuje novi element integralne slike. Cena faze je dva takta po pikselu, odnosno 16.200 taktova za segment 90×90 .

5.3. Faza B: šablon i njegova srednja vrednost

Šablon se čita linearno, po istom obrascu adresa/podatak, uz akumulaciju sume. Po završetku se pokreće sekvencijalni delilac koji računa $\text{template_mean} = (\text{sum_t} + N/2) / N$, gde je $N = \text{tmp_w} \cdot \text{tmp_h}$. Deljenje sa zaokruživanjem je namerno: zaokružena celobrojna srednja vrednost je preduslov za bit-egzaktan datapath (odjeljak 2.5). Ova faza se izvršava jednom po pozivu.

5.4. Faza C: priprema prozora

Za svaku poziciju prozora treba pročitati četiri elementa integralne slike. Pošto je integralna slika realizovana kao blok-memorija sa jednim portom, četiri čitanja **ne mogu** biti istovremena — razložena su u pet stanja (`S_L_U_A` do `S_L_U_E`), pri čemu svako stanje izdaje sledeću adresu i istovremeno akumulira podatak koji je stigao po prethodnoj. U petom stanju se, uz poslednji sabirak, pokreće delilac za `f_bar` i nuliraju sva tri akumulatora, a `S_L_U_WAIT` čeka rezultat deljenja.

5.5. Faza D: pipeline-ovana MAC petlja

Ovo je unutrašnja petlja i ubedljivo najskuplji deo obrade: izvršava se $\text{tmp_w} \cdot \text{tmp_h}$ puta po svakoj poziciji prozora, dakle preko 3,3 miliona puta po pozivu za pun segment. Naivna realizacija bi po pikselu trošila dva takta (jedan za adresu, jedan za podatak). Umesto toga, faza je organizovana protočno, u **tri stepena**: u stanju `S_L_YX_RUN` se **istovremeno** izdaje adresa piksela $k + 2$, registruju razlike df/dt piksela $k + 1$, i akumulira već registrovani par piksela k (u `sum_num`, `sum_den_f`, `sum_den_t`). Stanje `S_L_YX_FILL` puni protočnu strukturu (izdaje samo prvu adresu, bez registrovanja i akumulacije), a `S_L_YX_DRAIN` i `S_L_YX_DRAIN2` su prazna za izdavanje adrese — samo dovršavaju registrovanje i akumulaciju piksela koji su već učitani. Presek je uveden da bi se putanja $\text{BRAM} \rightarrow \text{oduzimanje} \rightarrow \text{množenje} \rightarrow \text{akumulator}$ razdvojila na dva takta; sam je doneo **2,43 ns** vremenske rezerve (odjeljak 10.5). Cena je i dalje **jedan takt po pikselu**, uz dva takta režije po prozoru umesto jednog. Slika 2 prikazuje preklapanje.

takt	t0	t1	t2	t3	...	tN-1	tN	tN+1
stanje	FILL	RUN (ponavlja se N-1 puta)				RUN	DRAIN	DRAIN2
izdaje adresu	px 0	px 1	px 2	px 3	...	px N-1	—	—
registruje df/dt	—	px 0	px 1	px 2	...	px N-2	px N-1	—
akumulira	—	—	px 0	px 1	...	px N-3	px N-2	px N-1

Ukupno: $N + 2$ takta za N piksela (jedan takt po pikselu + dva takta režije).

Bez protočnosti bi bilo $2N$ taktova — ušteda blizu 50 % ukupne latencije jezgra.

Slika 2. Trostepena protočnost unutrašnje petlje: preklapanje izdavanja adrese, registrovanja razlika df/dt i akumulacije

5.6. Faza E: normalizacija i upis rezultata

U stanju S_NCC_SQ se najpre proverava degenerisani slučaj: ako je bilo koja od suma kvadrata jednaka nuli, rezultat je 0 i deljenje se preskače. Inače se registruju $num_sq \leq sum_num^2$ i $den_prod \leq sum_den_f \cdot sum_den_t$. Stanje S_NCC_DIV pomera tako registrovan deljenik za 31 mesto ulevo i pokreće 83-bitni delilac nad registrovanim vrednostima. Stanje S_NCC_WAIT čeka njegov signal završetka, a S_WRITE_RESULT upisuje rezultat u mapu i pomera brojače prozora. Kvadriranje je razdvojeno od pokretanja delioca u dva takta u Koraku 8b: putanja od kvadriranja sum_num do 83-bitnog delioca bila je najgora posle rutiranja, pa je presečena registrom.

6. Datapath i controlpath

6.1. Registri

Registri su identifikovani iz oblika `if-goto` (poglavlje 3), grupisanjem po odredišnoj promenljivoj. Tabela 6 daje pregled.

Registar	Širina	Upisuje se u fazi
<code>img_w, img_h, tmp_w, tmp_h, res_w, res_h</code>	8	S_IDLE (na start)
<code>y, x</code> — deljeni: izgradnja SAT i unutrašnja petlja	7	A i D
<code>v, u</code> — pozicija prozora	7	C i E
<code>p</code> — linearni indeks šablona	10	B
<code>row_sum, sum_t, sum_f_partial</code>	32	A, B, C
<code>template_mean, f_bar</code>	8	B, C
<code>df_reg, dt_reg</code> — registrovane razlike piksel-sredina (MAC stepen 1, Korak 8b)	9	D
<code>mac_v_reg</code> — validnost stepena 1 (gejtuje akumulaciju u taktu punjenja)	1	D
<code>sum_num</code> (označen)	27	D (akumulator)
<code>sum_den_f, sum_den_t</code>	26	D (akumulatori)
<code>num_sq_reg, den_prod_reg</code> — registrovani ulazi 83-bitnog delioca (Korak 8b)	52	E
<code>result_q</code>	32	E
<code>busy, done</code>	1	S_IDLE, S_DONE

Tabela 6. Registri datapath-a

6.2. Funkcionalne jedinice

Datapath sadrži tri paralelna množača u unutrašnjoj petlji ($df_reg \cdot dt_reg$, df_reg^2 , dt_reg^2), koji se preslikavaju u DSP48E1 blokove, kao i dva množača van petlje (kvadriranje brojioca i proizvod imenilaca). Deljenje resursa nije primenjeno na tri množača unutrašnje petlje: pošto se ta petlja izvršava milionima puta, serijalizacija bi utrostručila najskuplji deo obrade radi uštede od svega nekoliko DSP blokova, kojih ionako ima dovoljno (odeljak 7.2).

Pet registara iz tabele iznad (`df_reg`, `dt_reg`, `mac_v_reg`, `num_sq_reg`, `den_prod_reg`) nisu deo prvobitnog dizajna — uvedeni su u Koraku 8b isključivo radi zatvaranja vremenskih uslova, bez ikakve promene izlaza. Njihova cena je **+0,41 %** latencije i **+110 flip-flopova**, a dobitak **2,59 ns** vremenske rezerve (odeljak 10.5).

6.3. Sekvencijalni delilac

Deljenje promenljivim deliocem je najskuplja operacija u datapath-u. Realizovano je kao zaseban parametrizovan modul `seq_divider`, po klasičnom algoritmu sa vraćanjem (engl. *restoring division*): u svakom taktu se ostatak pomera za jedno mesto, poredi sa deliocem i po potrebi umanjuje, čime se dobija jedan bit količnika. Kritična putanja po taktu je time svedena na jedno oduzimanje i jedno poređenje.

Modul se instancira dva puta, sa različitim parametrom širine:

Instanca	Širina	Namena	Taktova po deljenju	Poziva po pozivu jezgra
div_mean	18	template_mean i f_bar	~19	1 + res_w·res_h
div_ncc	83	$NCC^2 = (\text{num_sq} \ll 31) / \text{den_prod}$	~83	res_w·res_h

Tabela 7. Instance sekvencijalnog delioca

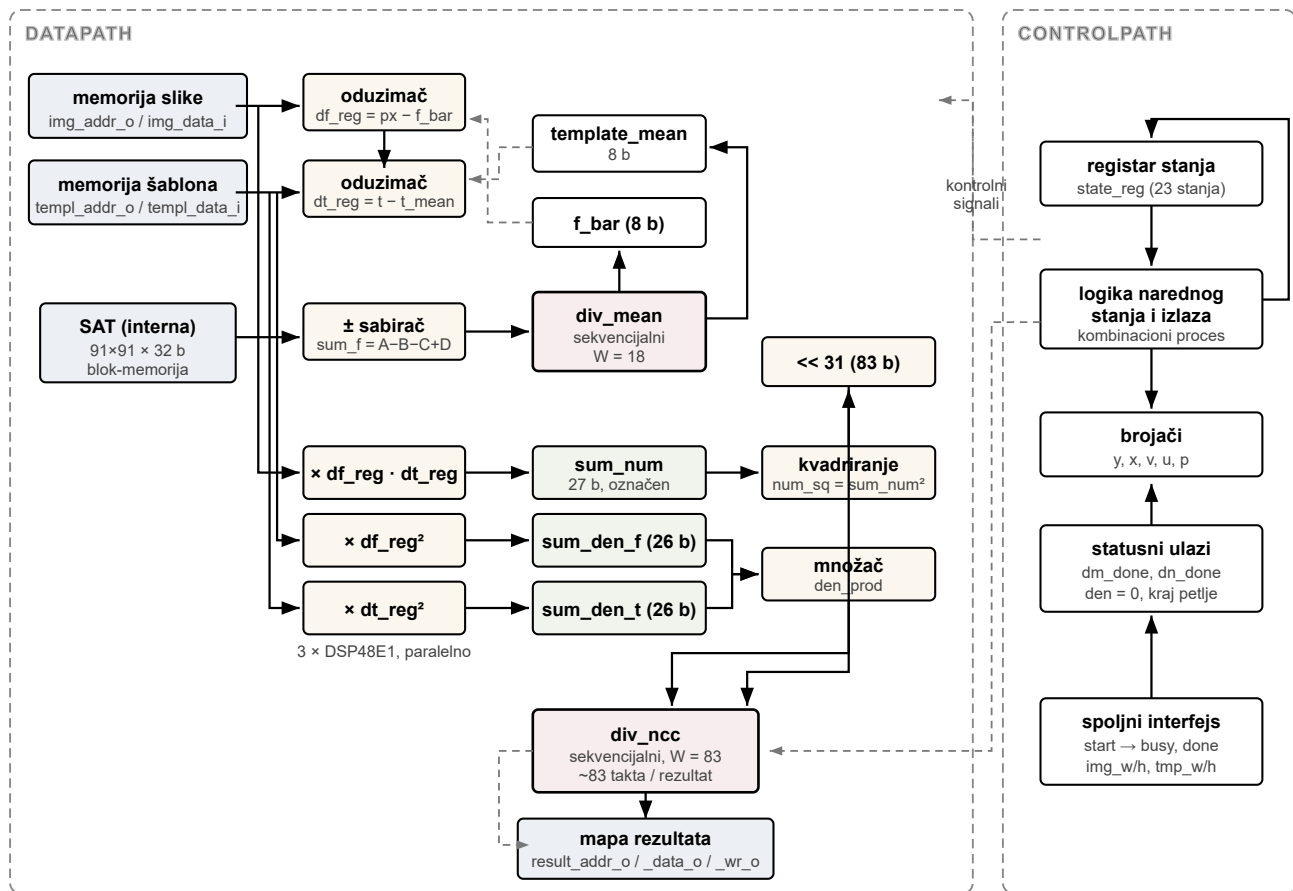
Instanca div_mean deljena je između dve namene (srednja vrednost šablona i srednja vrednost prozora) jer se te dve operacije nikada ne izvršavaju istovremeno — primer deljenja resursa koje ništa ne košta. Komunikacija sa deliocem je preko rukovanja: kontrolna jedinica postavlja operande i generiše impuls start, zatim čeka signal done u posebnom stanju čekanja.

6.4. Memorije

Memorija	Veličina	Pristup	Realizacija
segment slike	$90 \times 90 \times 8 \text{ b}$	čitanje	spoljna, deljena (procesor / DMA / oba jezgra)
šablon	$30 \times 30 \times 8 \text{ b}$	čitanje	ista spoljna memorija, druga adresna oblast
integralna slika (SAT)	$91 \times 91 \times 32 \text{ b}$	čitanje i upis	interna , privatna, blok-memorija
mapa rezultata	do $90 \times 90 \times 32 \text{ b}$	upis (jezgro), čitanje (procesor)	spoljna, offset ADDR_RESULTS

Tabela 8. Memorije u sistemu

6.5. Blok dijagram



Slika 3. Blok dijagram datapath-a i controlpath-a NCC jezgra

Podela je klasična: controlpath je isključivo mašina stanja sa brojačima, koja prema datapath-u šalje upravljačke signale (izbor operanada, dozvole upisa, impulse za pokretanje delilaca), a od njega prima statusne signale (završetak deljenja, nulta varijansa, kraj petlje). Datapath ne sadrži nikakvu odluku o toku izvršavanja.

Radi preglednosti, slika ne iscertava zasebne blokove za pet protočnih registara iz Koraka 8b (df_reg/dt_reg/mac_v_reg između oduzimača i množača, num_sq_reg/den_prod_reg između kvadriranja i delioca) — oni takt-granicu unose unutar prikazanih putanja, bez promene redosleda blokova ni toka podataka (Tabela 6, §6.2).

7. Analiza posle sinteze i implementacije

7.1. Metod

Analiza je sprovedena nad opisom u fajlovima `ncc_pkg.vhd` i `ncc_core.vhd`, koji zajedno čine ceo dizajn (modul `seq_divider` nalazi se u istom fajlu kao i jezgro). Ovo poglavlje analizira **samostalno jezgro**; analiza celog integrisanog sistema je u poglavlju 10.

- **Sinteza, implementacija i vremenska analiza:** Vivado 2025.2, `synth_design -mode out_of_context -top ncc_core -part xc7z010clg400-1`, pa pun tok implementacije (`opt_design`, `place_design`, `phys_opt_design`, `route_design`, `phys_opt_design`) i izveštaji `report_utilization` i `report_timing_summary`. Sve brojke u ovom poglavlju su **post-route**, ne post-sintezne.
- **Funkcionalna provera i merenje latencije:** XSim, simulacija ponašanja nad stvarnim podacima; `testbench` broji takte u kojima je signal `busy` aktivan.

Dve metodološke odluke koje utiču na brojke. Prvo, ograničenje takta se čita **pre** `synth_design`-a: ako se `create_clock` izda posle sinteze, jezgro se sintetiše bez vremenskog cilja, pa je svaka izmerena rezerva merena nad kolom od koga zadata frekvencija nikad nije ni tražena. Drugo, koristi se `-mode out_of_context`: golo jezgro ima 118 portova, a paket `clg400` nudi 100 korisničkih pinova, pa bi bez tog režima alat procenjivao rutiranje nad nemogućim razmeštajem.

7.2. Utrošeni resursi

Resurs	Utrošeno	Raspoloživo	Iskorišćenost
Slice LUT (sve kao logika)	1.472	17.600	8,36 %
Slice registri (FF)	664	35.200	1,89 %
DSP48E1	9	80	11,25 %
Block RAM (RAMB36E1)	9	60	15,00 %
LUT kao memorija	0	6.000	0,00 %

Tabela 9. *Utrošeni resursi samostalnog jezgra posle implementacije (xc7z010clg400-1, post-route)*

Jezgro zauzima manje od devetine logičkih resursa uređaja. Red *LUT kao memorija* je jednak nuli, što je potvrda da je integralna slika stvarno smeštena u blok-memoriju, a ne raspodeljena po LUT-ovima — to je bio cilj, jer bi distribuirana memorija te veličine sama pojela ceo budžet logike.

Sistemska model predviđa **dve** paralelne instance, a pored njih u programabilnoj logici moraju stati i AXI interkonekt i DMA. Ovde je važna razlika u tome *šta* se meri: brojke iz Tabele 9 su **golo jezgro**, dok je u integrisanom sistemu jedna instanca ceo IP `ncc_accel` — jezgro *plus* dva AXI kontrolera i memorijski podsistem — što po instanci iznosi 1.910 LUT, ne 1.472. Izmereno zauzeće celog sistema je **35,6 %** LUT-ova (odjeljak 10.2), i time staje sa rezervom.

7.3. Kritična putanja i maksimalna frekvencija

Veličina	pri 11,000 ns	pri 10,000 ns
Zadata frekvencija	90,909 MHz	100 MHz
Najgora vremenska rezerva (WNS)	+0,387 ns	+0,146 ns
Kašnjenje kritične putanje	10,638 ns (logika 5,719 · rutiranje 4,919)	9,832 ns (logika 7,901 · rutiranje 1,931)
Broj nivoa logike	16 (9×CARRY4, LUT)	12 (10×CARRY4, 2×DSP48E1)
Krajnjih tačaka koje krše ograničenje	0 od 1.804	0
Maksimalna frekvencija		~101,5 MHz

Tabela 10. Rezultati vremenske analize samostalnog jezgra (post-route, xc7z010clg400-1), pri dva zadata perioda takta

Samostalno jezgro zatvara i ograničenje od 100 MHz, sa rezervom od 0,146 ns; ostvariva frekvencija je približno 101,5 MHz. Nijedna krajnja tačka ne krši ograničenje ni pri jednom od dva perioda.

Zašto su navedena dva merenja. Alat optimizuje *do* zadatog ograničenja i staje čim ga ispuni, pa je kritična putanja pri labavijem ograničenju *duža*: na 11 ns iznosi 10,638 ns sa 46 % rutiranja, a na 10 ns svega 9,832 ns sa 20 % rutiranja — jer je alat pri strožem cilju kvadriranje mapirao u DSP48E1 umesto u lanac prenosa. To je isti RTL u dve različite implementacije. Vremenska rezerva se zato **ne sme aritmetički prevoditi** između ograničenja, a maksimalna frekvencija se navodi na osnovu merenja na onom ograničenju koje se tvrdi.

Kritična putanja vodi od akumulatora sum_num_reg do registra num_sq_reg, dakle kroz kvadriranje brojioca. Ona se izvršava **jednom po poziciji prozora**, a ne u unutrašnjoj petlji — zato je presek registrom na tom mestu (odjeljak 5.6) koštao samo jedan takt po prozoru, oko 0,2 % latencije. Nakon tog preseka putanja kroz 83-bitni delilac više nije vezujuća; u integrisanom sistemu vezujuća postaje adresna putanja unutar jezgra, i to tek pod pritiskom razmeštaja (odjeljak 10.3).

Ovaj rezultat je posledica jedne projektne odluke: sva tri deljenja izvode se sekvencijalno, jedan bit količnika po taktu. Kombinaciono deljenje promenljivim deliocem davalo bi vremensku rezervu od približno –46 ns, dakle sintetizovano kolo ne bi radilo ni na 20 MHz. Sekvencijalna realizacija svodi kritičnu putanju delioca na jedno oduzimanje i jedno poređenje, po cenu latencije koja se, kao što pokazuje sledeći odeljak, u ukupnom bilansu jedva primećuje.

7.4. Latencija i propusnost

Latencija je izmerena simulacijom na stvarnim podacima: gornji levi segment 90×90 slike board2.txt (polje a8 šahovske table) i šablon crnog topa dimenzija 25×15. Mapa rezultata je 66×76, dakle 5.016 pozicija prozora.

Veličina	Vrednost
Latencija po pozivu (izmereno)	2.461.201 taktova
Broj obrađenih pozicija prozora	5.016
Taktova po piksel-operaciji	1,31
Vreme po pozivu pri 100 MHz	24,61 ms
→ propusnost	~203.800 pozicija/s (4,91 μs po poziciji)
Vreme po pozivu pri 90,909 MHz	27,07 ms
→ propusnost	~185.300 pozicija/s (5,40 μs po poziciji)

Tabela 11. Izmerena latencija i izvedena propusnost

Zašto dve frekvencije. Latencija u taktovima je svojstvo dizajna i ne zavisi od takta; vreme zavisi. Samostalno jezgro zatvara 100 MHz (odjeljak 7.3), ali u integrisanom sistemu takt je **90,909 MHz**, jer PLL procesorskog sistema za traženih 95 MHz daje 1000/11 (odjeljak 10.3). Navedene su zato obe brojke: 24,61 ms je ono što jezgro može samo, 27,07 ms je ono što sistem stvarno radi.

Terminologija. Pod *latencijom* se podrazumeva broj taktova od postavljanja signala start do signala done za jedan poziv jezgra; pod *propusnošću* količina posla po jedinici vremena, ovde izražena brojem obrađenih pozicija prozora u sekundi. Pošto jezgro obrađuje jedan poziv u potpunosti pre nego što prihvatiti sledeći, ove dve veličine su uzajamno recipročne. Konkretno, pri 90,909 MHz propusnost = 5.016 pozicija / 27,07 ms ≈ 185.300 pozicija/s (odnosno 1 / 5,40 μs), a takta po piksel-operaciji = 2.461.201 / (5.016 · 375) = 1,31.

Izmerena latencija se dobro poklapa sa analitičkim modelom. Trošak po poziciji prozora iznosi:

$$T_{prozor} = tmp_w \cdot tmp_h + 112 \text{ taktova}$$

gde sabirak 112 čine: pet taktova za čitanje integralne slike, oko 19 taktova za deljenje pri računanju f_bar , **tri** takta za punjenje i pražnjenje protočne strukture (FILL, DRAIN, DRAIN2), **jedan** takt za registrovanje num_sq i den_prod u stanju S_NCC_SQ, oko 83 takta za 83-bitno deljenje i jedan takt za upis rezultata. Poslednja dva sabirka su posledica preseka iz Koraka 8b (odjeljak 10.5); pre njega je sabirak bio 110. Za šablon 25×15 to je 487 taktova po prozoru, odnosno 2.442.792 takta za svih 5.016 pozicija. Uz fazu A (16.200) i fazu B (769), model predviđa 2.459.761 takt, dok je izmereno 2.461.201 — odstupanje je 1.440 taktova, odnosno **0,06 %**. Model se, dakle, može koristiti za predviđanje latencije pri drugim veličinama segmenta i šablona.

Ukupna latencija je zbir tri člana:

Faza	Model	Taktova	Udeo
A — učitavanje slike i izgradnja SAT	$2 \cdot img_w \cdot img_h = 2 \cdot 90 \cdot 90$	16.200	0,7 %
B — učitavanje šablona i njegova srednja vrednost	$2 \cdot N + 19 \text{ (} N = 375 \text{)}$	769	< 0,1 %
C, D, E — glavna petlja	$res_w \cdot res_h \cdot (N+112) = 66 \cdot 76 \cdot 487$	2.442.792	99,3 %
ostatak (prelazi S_L_V, zaokruženja modela)	—	1.440	0,1 %
Ukupno (izmereno)	—	2.461.201	100 %

Tabela 12. Raščlanjenje latencije po fazama (šablon 25×15)

Dominacija glavne petlje potvrđuje da je protočnost unutrašnje petlje bila prava meta optimizacije: ona neposredno određuje član N u izrazu iznad, koji za realne veličine šablona nosi 77 % troška po prozoru.

7.5. Funkcionalna verifikacija

Ispravnost je proverena na **dva nivoa** — nad golim jezgrom i nad spakovanim IP-om kroz AXI — a svi testovi porede rezultat sa referentnim vrednostima iz validiranog modela u jeziku C (Korak 1), koji je nezavisno proveren nad celom tablom: 32 od 32 figure ispravno prepoznate, uz FEN notaciju identičnu zvaničnoj.

Poređenje između nivoa je samo po sebi rezultat: IP daje **bit-identičan** izlaz kao golo jezgro, što znači da AXI omotač ne unosi nikakvu izmenu u obradu. Isti pik $0x80000000$ pojavljuje se na linearnom indeksu 956, a to je tačno pozicija ($u = 32$, $v = 14$) jer je $14 \cdot 66 + 32 = 956$.

Test	Podaci	Očekivano	Ishod
ncc_core_tb	sintetički segment 4×4 , šablon 2×2	svih 9 pozicija prozora bit-identično referentnom modelu, uključujući oba ekstrema ($0x00000000$ i $0x80000000$)	prolazi (9 / 9)
ncc_core_real_tb	stvarni segment 90×90 (polje a8) i šablon crnog topa 25×15	maksimum $0x80000000$ na poziciji ($u = 32$, $v = 14$)	prolazi ($0x80000000$ @ 32,14)
ncc_accel_tb	isti stvarni podaci, ali upisani i pročitani kroz AXI (Lite za registre, Full za memorije)	isti maksimum, na linearnom indeksu 956 — dokaz da je omotač transparentan	prolazi ($0x80000000$ @ idx 956)
ncc_accel_burst_tb	AXI burst dužine 8 ($awlen/arlen = 7$), proverava WSTRB i ranog AW	svih 8 beat-ova ispravno u oba smera; upis sa isključenim bajt-lane-om ne menja piksel	prolazi (8 / 8 čitanje i upis)

Tabela 13. Rezultati funkcionalne verifikacije

Zašto postoji poseban test za burst-ove. Prvi integracioni testbench koristio je isključivo pojedinačne transfere ($len = 0$), pa je jedan defekt u generisanom AXI kontroleru ostao neotkriven: pri čitanju burst-om vraćala se *prethodna* reč na svakom beat-u posle prvog. Procesor taj put nikad ne pogodi, jer su $Xi1_In32/Xi1_Out32$ pojedinačni transferi — ali svaki DMA prenos ga pogodi. Test je zato napisan tako da **prvo padne** nad starom logikom, pa prođe nad ispravljenom.

Sintetički test je namerno konstruisan tako da šablon bude tačno ugrađen u sliku na poziciji (1, 1): time se u istom testu pokrivaju i tačna nula i tačan maksimum 2^{31} , a svih devet vrednosti se može proveriti ručno. Test nad stvarnim podacima potvrđuje ponašanje na punoj veličini problema i na podacima koji nisu konstruisani: dobijeni maksimum $0x80000000$ odgovara vrednosti $NCC^2 = 1,0$, što je daleko iznad praga 0,5625 ($0x48000000$), a položaj maksimuma odgovara polju koje u zvaničnoj FEN notaciji zaista sadrži crnog topa.

8. Pakovanje u IP jezgro

8.1. Izbor obrasca: slave sa internim memorijama

Da bi jezgro postalo upotrebljivo u alatu za integraciju blokova, mora dobiti standardni interfejs. Postoje dva uobičajena obrasca. U prvom je akcelerator **master** na magistrali i sam čita i piše po deljenoj memoriji; u drugom je **slave** sa sopstvenim internim memorijama, u koje procesor ili DMA prethodno prepisu podatke. Izabran je drugi.

Razlog je praktičan. Master interfejs zahteva sopstvenu mašinu stanja za adresiranje, rukovanje sa `arvalid/rready` i obradu odgovora, dakle novu, neverifikovanu logiku u bloku koji je do tada bio bit-tačan. Slave obrazac ne dira jezgro uopšte: `ncc_core` ulazi u omotač **nepromenjen**, sa istim adresno-podatkovnim portovima kojima je i pre pristupao memoriji. Verifikacija iz odeljka 7.5 time ostaje na snazi, a to je i potvrđeno merenjem — izlaz kroz AXI je bit-identičan.

Cena te odluke je da segment slike i šablon moraju biti *preneseni* u interne memorije pre svakog poziva, umesto da ih jezgro čita direktno iz deljene memorije. Koliko to košta, pokazuje odeljak 9.5: oko 7 % ukupnog vremena ako prenos radi procesor, oko 1 % ako ga radi DMA.

Odstupanje od sistemskog modela. Sistemski model iz prethodne faze projekta pretpostavlja deljenu blok-memoriju na adresi `0x4000_0000` i registre `REG_IMG_ADDR/REG_TMP_ADDR` kojima procesor javlja gde se podaci nalaze. U slave obrascu deljene memorije nema, pa ta adresa **ne postoji**, a ta dva registra postaju **rezervisana** — zadržana su u mapi da adrese ostalih registara ostanu nepromenjene.

8.2. Dva AXI interfejsa

Blok izlaže dva odvojena slave interfejsa, jer se saobraćaj prirodno deli na dve vrste: nekoliko kratkih pristupa registrima i masovni prenos piksela.

Interfejs	Tip	Širina adrese	Opseg	Uloga
S00_AXI	AXI4-Lite	6 bita	4 KB	kontrolni i statusni registri
S01_AXI	AXI4-Full	17 bita	128 KB	interne memorije: slika, šablon, rezultat

Tabela 14. AXI interfejsi IP bloka `ncc_accel`

AXI4-Lite je za registre dovoljan: pristupi su pojedinačni i retki. Za memorije je uzet pun AXI4, jer on podržava *burst* prenose — bez njih bi DMA morao da izdaje po jednu transakciju na svaku reč, što bi prenos od 8.100 reči učinilo besmisleno skupim.

Širina adrese S01 nije slobodan parametar. Port `mem_addr_o` ka memorijskom podsistemu je fiksno 17-bitni, pa je 17 jedina ispravna vrednost `C_S01_AXI_ADDR_WIDTH`. Pošto čarobnjak za pakovanje ume da tu vrednost vrati na svoju podrazumevanu, u kodu stoji statička tvrdnja koja sintezu **zaustavlja** ako vrednost nije 17 — umesto da blok tiho dekodira samo prvi kilobajt i preklopi sva tri regiona jedan preko drugog.

8.3. Regioni unutar memorijskog interfejsa

Sve tri memorije dele isti AXI interfejs, a razlikuju se po gornja dva bita adrese. Dekodovanje je čisto kombinaciono: `region = addr(16:15)`, indeks reči = `addr(14:2)`.

Region	addr(16:15)	offset	Sadržaj	Organizacija
Slika	00	+0x00000	pikseli segmenta	8 bita × 8.192
Šablon	01	+0x08000	pikseli šablona	8 bita × 1.024
Rezultat	10	+0x10000	NCC ² u formatu Q1.31	32 bita × 8.192

Tabela 15. *Regioni unutar S01_AXI (offset od bazne adrese bloka)*

Jedan piksel zauzima celu 32-bitnu reč, iako mu je dovoljan jedan bajt. To je namerno: procesor i DMA time pristupaju svim regionima na isti način, bez pakovanja i raspakivanja bajtova, a upis se svodi na prostu petlju. Cena je četiri puta veći prenos kroz magistralu, ali prenos nije usko grlo (odjeljak 9.5). Upis poštuje signale izbora bajtova, pa upis sa isključenim najnižim bajt-lane-om ne menja piksel.

Regioni su poravnati na 32 KB, a najduži burst je 1 KB, pa nijedan burst ne može preći iz jednog regiona u drugi — pod uslovom da softver ne pokrene prenos duži od samog regiona.

8.4. Kontrolni i statusni registri

Bazne adrese registara su namerno zadržane identične konstantama iz sistemskog modela, tako da softver iz prethodne faze projekta može biti prenesen bez menjanja adresa.

offset	Registar	Pristup	Značenje
0x00	IMG_W	upis/čitanje	širina segmenta slike
0x04	IMG_H	upis/čitanje	visina segmenta slike
0x08	TMP_W	upis/čitanje	širina šablona
0x0C	TMP_H	upis/čitanje	visina šablona
0x10	IMG_ADDR	—	rezervisan (videti 8.1)
0x14	TMP_ADDR	—	rezervisan (videti 8.1)
0x30	CTRL	upis	upis vrednosti sa bitom 0 = 1 pokreće obradu
0x34	STATUS	čitanje	bit 0 = done (pamti se), bit 1 = busy

Tabela 16. *Registri dostupni preko S00_AXI*

Dva detalja u ovoj tabeli su posledica razlike između hardvera i sistemskog modela, i oba su bitna za softver.

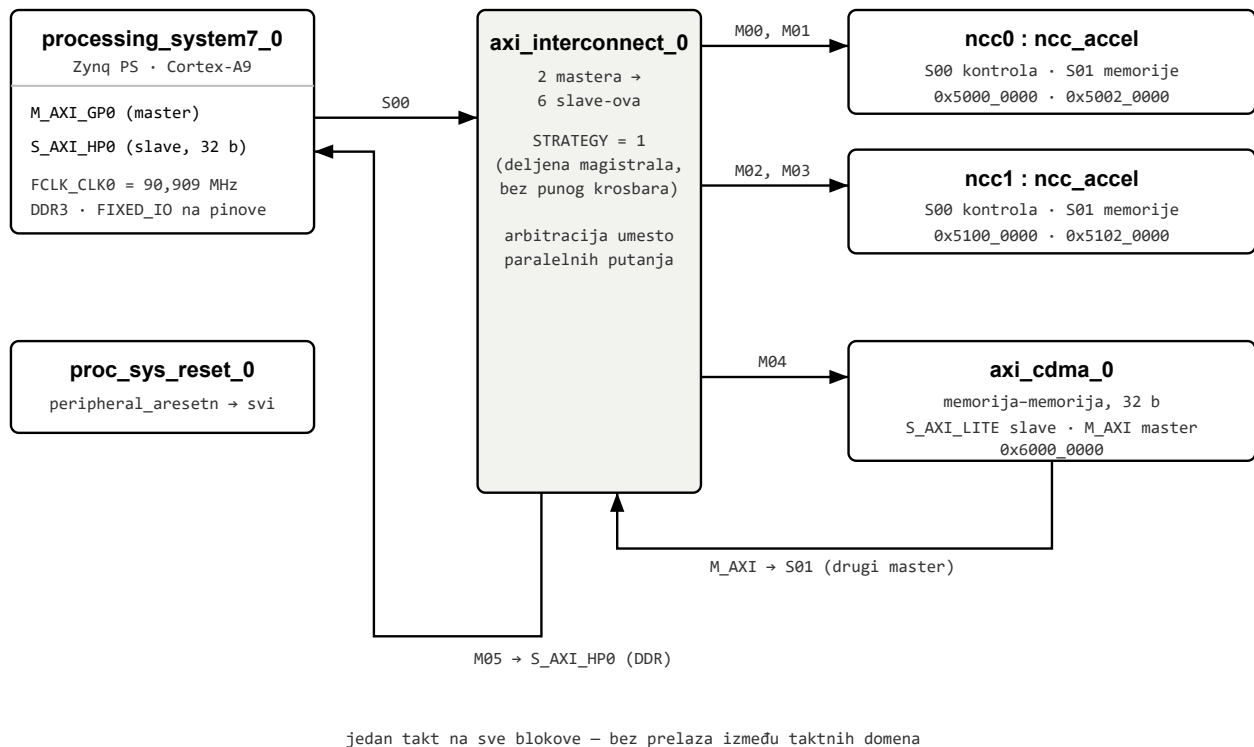
Start je impuls, ne stanje. Upis u CTRL proizvodi **jednotaktni** impuls start, jer jezgro tako očekuje. Da je umesto toga bio zadržan nivo, jezgro bi po završetku obrade odmah krenulo ponovo.

Signal done se pamti. Jezgro ga daje kao jednotaktni impuls, koji bi procesor koji povremeno čita status gotovo sigurno propustio. Zato ga omotač pamti u bistabilu, koji se briše na sledeći start. Bit busy se prenosi neizmenjen. Ovim se razlikuje od sistemskog modela, gde je isti registar imao vrednost 0 za *zauzeto* i 1 za *gotovo* — ovde su to dva odvojena bita, jer se stanja *još nije počelo* i *u toku* inače ne mogu razlikovati.

9. Integracija u sistem

9.1. Topologija

Sistem `ncc_system` sastavljen je od procesorskog sistema Zynq, **dve** instance akceleratora, jednog DMA kontrolera za prenos memorija-memorija i jednog AXI interkonekta koji ih sve povezuje. Ceo sistem radi na **jednom taktu**, pa u njemu nema prelaza između taktnih domena i nije potrebno nijedno ograničenje za lažne putanje.



Slika 4. Topologija integrisanog sistema `ncc_system`: Zynq PS, dve instance akceleratora, DMA memorija–memorija i jedan AXI interkonekt

Podešavanja DMA kontrolera. Kontroler radi u **prostom režimu**, bez raspršenog prikupljanja (*scatter-gather*): svaki prenos se programira upisom izvorne adrese, odredišne adrese i dužine, i pokreće pojedinačno. Lista deskriptora bila bi korisna kad bi se nizalo mnogo malih prenosa, a ovde ih je po polju table svega nekoliko, i to velikih. Isključeno je i poravnavanje podataka (*data realignment*), jer su svi pristupi poravnati na četiri bajta — jedan piksel po reči (odjeljak 8.3). Širina podataka je 32 bita, kao i sve ostalo u sistemu, a najveća dužina bursta je 256 prenosa, dakle 1 KB — ispod granice od 4 KB koju AXI postavlja za jedan burst, i dovoljno malo da nijedan burst ne pređe iz jednog regiona memorije u drugi.

Zašto jedan interkonekt, a ne dva. AXI slave interfejs može biti povezan na **samo jedan** interkonekt. Pošto memorijskom interfejsu `S01` moraju pristupati i procesor i DMA, oba moraju ulaziti u isti interkonekt — odatle konfiguracija sa dva mastera i šest slave-ova.

Zašto deljena magistrala, a ne pun krosbar. Prva verzija koristila je interkonekt koji uvek gradi pun krosbar sa protočnim stepenom na svakom portu. U konfiguraciji 2×6 to je trošilo **8.673 LUT** — skoro polovinu celog budžeta logike uređaja — i obaralo vremenska ograničenja. Zamenom deljenom

magistralom sa arbitracijom zauzeće je palo na **1.616 LUT**, dakle na manje od petine. Za ovaj saobraćaj je to dovoljno, jer se dva mastera ionako serijalizuju u softveru: procesor čeka da DMA prenos završi pre nego što nastavi.

Zašto je port S_AXI_HP0 sužen na 32 bita. Portovi visokih performansi na Zynq-u su nativno 64-bitni, jer su predviđeni za širok saobraćaj ka DDR-u. Sve ostalo u ovom sistemu je 32-bitno. Interkonekt širinu krosbara određuje po *najširem* portu, pa je zbog tog jednog porta gradio 64-bitnu magistralu i lepio konvertor širine na **svaki** 32-bitni port — šest konvertora za konverziju koju nijedan blok nije tražio. Sužavanjem porta na 32 bita svi su nestali: **3.334 LUT i 3.659 flip-flopova manje**. Propusnost se pri tome ne gubi, jer je DMA kontroler 32-bitni i širinu od 64 bita nikad nije mogao ni iskoristiti.

9.2. Adresna mapa

Mapa je fiksirana u skripti koja gradi sistem, a ne prepuštena automatskom dodeljivanju. Time se dobija da rekonstrukcija sistema uvek daje iste adrese, što je uslov da softver iz prethodne faze projekta radi bez izmena.

Master	Slave	Bazna adresa	Opseg
PS M_AXI_GP0	ncc0.S00_AXI (kontrola)	0x5000_0000	4 KB
	ncc0.S01_AXI (memorije)	0x5002_0000	128 KB
	ncc1.S00_AXI	0x5100_0000	4 KB
	ncc1.S01_AXI	0x5102_0000	128 KB
	axi_cdma_0.S_AXI_LITE	0x6000_0000	64 KB
DMA M_AXI	PS S_AXI_HP0 (DDR)	0x0000_0000	iz PS podešavanja
	ncc0.S01_AXI	0x5002_0000	128 KB
	ncc1.S01_AXI	0x5102_0000	128 KB

Tabela 17. Adresna mapa, po masteru

Vidljivost je namerno različita po masteru. DMA **ne vidi** kontrolne registre, jer mu ne trebaju — sužavanjem njegove mape smanjuje se šansa da pogrešno programiran prenos upiše nešto u registre. Procesor **ne vidi** DDR kroz programabilnu logiku, jer mu do sopstvene memorije vodi direktan put kroz procesorski sistem.

Mapa je namerno usklađena sa konstantama sistemskog modela iz prethodne faze projekta, da softver ne mora da menja adrese.

Konstanta modela	Vrednost	Preslikavanje u hardveru
ADDR_NCC	0x5000_0000	ncc0.S00_AXI — identično
ADDR_NCC1	0x5100_0000	ncc1.S00_AXI — identično
ADDR_DMA	0x6000_0000	axi_cdma_0.S_AXI_LITE — identično
ADDR_BRAM	0x4000_0000	ne postoji — nema deljene memorije (odjeljak 8.1)
ADDR_RESULTS	0x40	postaje S01 + 0x10000, dakle drugi interfejs
REG_IMG_ADDR	0x10	<i>rezervisan</i> — bez značenja u slave obrascu
REG_TMP_ADDR	0x14	<i>rezervisan</i>

Tabela 18. Poklapanje sa konstantama sistemskog modela

Opseg od 4 KB za kontrolne registre je najmanji koji alat dopušta za AXI segment; blok dekodira samo donjih šest bita, pa se registri unutar tog prozora ponavljaju, što je bezopasno.

9.3. Tok podataka po jednom polju table

Softver za svako neprazno polje šahovske table izvršava sledeći niz. Redosled je određen time što su memorije jednostruko baferovane, pa jezgro ne može računati dok se u njega upisuje.

1. Procesor iz slike u DDR-u proverava da li je polje prazno — bez ikakvog pristupa programabilnoj logici.
2. Procesor u DDR-u priprema bafer u kome je **jedan piksel po 32-bitnoj reči**, pa ga izbacuje iz keša, da bi DMA video tačne podatke.
3. DMA prenosi segment slike iz DDR-a u S01 + 0x00000, prvo jednog pa drugog akceleratora.
4. DMA prenosi šablon u S01 + 0x08000. Šabloni se raspakuju **jednom na početku programa**, ne po svakom polju.
5. Procesor upisuje dimenzije u registre, pa upisom u CTRL pokreće obradu na oba bloka.
6. Procesor u petlji čita STATUS i čeka bit done na oba bloka.
7. DMA prenosi mapu rezultata iz S01 + 0x10000 u DDR; procesor poništava keš nad tim opsegom i traži maksimum.

9.4. Dva pravila koja softver mora poštovati

Skorovi se porede kao neoznačeni brojevi. Vrednost 0x80000000 označava $NCC^2 = 1,0$, dakle savršeno poklapanje — ali tumačena kao označen 32-bitni ceo broj ona je **negativna**. Softver koji traži maksimum polazeći od -1 i koristi označeno poređenje odbacio bi upravo savršeno poklapanje. Na realnim podacima se to ne javlja, jer je prag odlučivanja 0,5625, ali je greška latentna i mora biti izbegnuta.

Nikad DMA i procesor nad istim memorijskim interfejsom istovremeno. Unutar bloka adresa se pri istovremenom čitanju i upisu vodi po čitanju, a upis nije zabranjen — pa bi se u tom slučaju upis obavio na adresu koja se čita. Situacija je sa jednim procesorom i jednim DMA kanalom nedostižna, jer se oni serijalizuju u softveru, i zato arbitracija nije ni ugrađena: bila bi mrtav hardver. Ali invarijanta mora biti zapisana, jer je hardver ne proverava.

9.5. Zašto DMA, iako nije performansni dobitak

DMA je u ovom sistemu izabran zato što ga zadatak izričito navodi među komponentama za integraciju, i zato što je popravka burst prenosa (odjeljak 7.5) ionako bila neophodna — a ne zato što donosi značajno ubrzanje. To treba reći otvoreno, sa brojkama.

Po jednom polju table prenosi se oko 146 KB. Ako prenos radi procesor pojedinačnim pristupima, to traje oko 3,4 ms, odnosno oko 7 % ukupnog vremena obrade jednog polja. DMA isti posao obavi za oko 0,4 ms, dakle oko 1 %. **Neto dobitak je oko 6 % ukupnog vremena.**

Preklapanje prenosa sa računanjem — što bi bio pravi dobitak — nije moguće ni u jednom od dva slučaja, jer su memorije jednostruko baferovane i jezgro ih čita dok računa. DMA je zato *brži prenos*, a ne *skrivena latencija*. Dvostruko baferovanje memorije slike bilo bi izmena u samom bloku i pravac za dalji rad.

10. Analiza integrisanog sistema

10.1. Metod

Ceo sistem se gradi i implementira **jednom skriptom, u komandnom režimu**, bez ručnih koraka u grafičkom okruženju. Skripta sastavlja sistem, pokreće sintezu i implementaciju do faze fizičke optimizacije posle rutiranja, i ispisuje izveštaje. Brojke u ovom poglavlju su sve **post-route**.

U tok su ugrađene dve **kapije** koje ga zaustavljaju na tihim otkazima na koje smo već naleteli. Prva proverava da IP nije sintetisan kao prazna kutija: ako se u paketu pogreši oznaka jezika, alat prestane da prepoznaje izvorne fajlove i napravi prazan blok **bez ijedne greške** — sinteza tada prođe u celini, a padne tek implementacija. Druga kapija poredi dve kopije izvora jezgra (jedna služi simulaciji, druga je unutar IP-a) i pada ako se razidu, jer bi se inače simulirao jedan a sintetisao drugi kod.

Fizička optimizacija nije opcija nego deo ugovora. Merenjem je utvrđeno da bez nje sistem **ne zatvara** vremenska ograničenja: standardna strategija daje rezervu od $-0,714$ ns, a strategija sa optimizacijom posle rutiranja $-0,233$ ns nad *istim* RTL-om. Skripta je zato izričito uključuje.

10.2. Utrošeni resursi

Resurs	Utrošeno	Raspoloživo	Iskorišćenost
Slice LUT	6.261	17.600	35,57 %
Slice registri (FF)	5.024	35.200	14,27 %
Block RAM (ukupno)	39	60	65,00 %
DSP48E1	18	80	22,50 %

Tabela 19. Resursi celog sistema (post-route, xc7z010clg400-1)

Blok	LUT	FF	RAMB36	DSP
ncc0 (ceo IP)	1.910	1.240	19	9
ncc1 (ceo IP)	1.910	1.240	19	9
axi_interconnect_0	1.616	—	—	—
axi_cdma_0	807	—	—	—

Tabela 20. Raspodela resursa po blokovima

Po instanci je **1.910 LUT**, a golo jezgro troši 1.472 (odeljak 7.2) — razlika od oko 440 LUT su dva AXI kontrolera i memorijski podsistem, dakle cena standardnog interfejsa. Interkonekt i DMA zajedno troše 2.423 LUT, oko 14 % uređaja.

Blok-memorija je najveće zauzeće i naša slabost. Sa 65 % je to jedini resurs bez velike rezerve, i tu smo **slabiji od referentnog rešenja** iz prethodne faze projekta, koje po instanci koristi svega četiri manja bloka. Uzrok je poznat i naveden je već u odeljku 7.2: reč integralne slike je 32-bitna, a dovoljno je **21 bit**, jer je najveća moguća suma $90 \cdot 90 \cdot 255 = 2.065.500$. Sužavanje te reči oslobodilo bi približno tri bloka po instanci, dakle oko šest ukupno, i spustilo zauzeće na oko 55 %. Nije izvedeno jer memorija nije usko grlo ovog dizajna, ali je to prva stvar koju treba uraditi ako se doda treća instanca.

10.3. Vremenska analiza i radni takt

Veličina	Vrednost
Radni takt	90,909 MHz (period 11,000 ns)
Najgora vremenska rezerva (WNS)	+0,170 ns
Najgora rezerva za zadržavanje (WHS)	+0,025 ns
Krajnjih tačaka koje krše ograničenje	0 od 15.459
Kritična putanja	ncc1/core_inst/y_reg[0] → ncc1/ms_inst/img_mem/ADDRBWRADDR[13]
Provera pravila projektovanja	bez kritičnih upozorenja

Tabela 21. Vremenska analiza sistema (post-route + fizička optimizacija)

Sistem zatvara vremenska ograničenja na 90,909 MHz, sa pozitivnom rezervom i za postavljanje i za zadržavanje, i bez ijedne krajnje tačke koja krši ograničenje.

Zašto 90,909 a ne 100 MHz. Traženo je 95 MHz, a PLL procesorskog sistema deli svoju ulaznu frekvenciju celim brojem, pa najbliža ostvariva vrednost iznosi $1000/11 = 90,909$ MHz. Odstupanje od nominalnih 100 MHz je **9,1 %**, dakle unutar granice od 20 % koju zadatak dopušta za razliku između sistemskog modela i realizacije.

Ograničenje je integracija, ne jezgro. Ovo treba reći precizno, jer je razlika bitna. Samostalno jezgro **zatvara** i 100 MHz, sa rezervom od 0,146 ns (odjeljak 7.3). Ceo sistem na istom ograničenju daje -0,232 ns. Kritična putanja u sistemu je adresna putanja *unutar* jezgra — ista logika koja samostalno nije ni blizu kritične — a postaje vezujuća tek pod pritiskom razmeštaja i zagušenja rutiranja u dizajnu koji zauzima trećinu uređaja. Dakle RTL je zadovoljavajući; cenu plaća integracija.

Poluga koja nije iskorišćena. Adresa piksela u unutrašnjoj petlji računa se u svakom taktu kao $(v + y) \cdot \text{img_w} + (u + x)$, dakle kroz množač. Mogla bi se računati **inkrementalno** — adresa u registru, uz korak +1 unutar reda i $+(\text{img_w} - \text{tmp_w} + 1)$ na prelasku reda — što izbacuje množač iz petlje i ne menja latenciju. Nije primenjeno iz dva razloga: dizajn već zadovoljava rubriku, a to bi bio treći zahvat u verifikovano jezgro. Uz to dobitak nije zagarantovan, jer je oko 62 % te putanje rutiranje, a ne logika — izbacivanje množača skraćuje samo logički deo.

10.4. Propusnost sistema

Latencija jednog poziva je izmerena simulacijom i iznosi 2.461.201 takt (odjeljak 7.4), što pri radnom taktu od 90,909 MHz daje **27,07 ms**. Model latencije omogućava da se ta brojka preračuna na drugu veličinu segmenta i šablona:

$$T = 2 \cdot \text{img_w} \cdot \text{img_h} + 2 \cdot N + \text{res_w} \cdot \text{res_h} \cdot (N + 112), \quad N = \text{tmp_w} \cdot \text{tmp_h}$$

Veličina zadatka	Taktova	Vreme	Izvor
segment 90×90, šablon 25×15	2.461.201	27,07 ms	izmereno simulacijom
segment 90×90, šablon 30×30	3.783.652	41,6 ms	iz modela

Tabela 22. Propusnost pri radnom taktu od 90,909 MHz

Pošto su u sistemu **dve** instance koje rade paralelno, sistem obrađuje dva polja table u istom vremenu u kome bi jedna instanca obradila jedno.

10.5. Put do zatvaranja vremenskih ograničenja

Prva verzija integrisanog sistema imala je vremensku rezervu od **−3,299 ns**, dakle nije ni blizu zatvarala. Do pozitivne rezerve se došlo kroz **pet** mera, navedenih po veličini doprinosa. Prve tri su popravile vreme, a poslednje dve površinu — razlika koja je objašnjena posle tabele.

Mera	Gde	Doprinos
Protočni stepen u MAC petlji	RTL jezgra	+2,43 ns
Fizička optimizacija posle rutiranja	tok implementacije	+0,481 ns
Registar pred deliocem	RTL jezgra	+0,155 ns
Deljena magistrala umesto krosbara	interkonekt	interkonekt sam: 8.673 → 1.616 LUT
Sužavanje porta S_AXI_HP0 na 32 bita	procesorski sistem	ceo sistem: 9.681 → 6.261 LUT 8.696 → 5.024 FF

Tabela 23. Mere kojima su zatvorena vremenska ograničenja

Dva zapažanja iz ove tabele vrede više od samih brojki.

Fizička optimizacija donosi više od drugog preseka u RTL-u. Njen doprinos od 0,481 ns je veći od 0,155 ns koje je dao registar pred deliocem — a ne zahteva nikakvu izmenu koda. To je razlog zašto je uključena u skriptu kao obavezan korak, a ne kao opcija.

Dve mere nisu popravile vreme, nego površinu — i to treba razlikovati. Zamena krosbara deljenom magistralom smanjila je sam interkonekt sa 8.673 na 1.616 LUT, a sužavanje porta spustilo je zauzeće celog sistema sa 55,0 % na 35,6 % (9.681 → 6.261 LUT), jer je uklonilo šest konvertora širine. Ali izmerena vremenska rezerva se pri sužavanju porta pomerila samo sa −0,233 na −0,232 ns, dakle praktično nimalo.

Prvobitna pretpostavka da kritična putanja ide kroz te konvertore bila je **netačna**, i vredi reći zašto je nastala: u jednom merenju konvertor se pojavio kao najgora putanja, ali samo zato što je na tom ograničenju adresna putanja imala rezervu. Uklanjanjem konvertora pokazalo se da je vezujuća putanja sve vreme bila adresna, unutar jezgra. Izmena je zadržana zbog površine, ne zbog vremena. Pouka je metodološka: pre pripisivanja uzroka treba uporediti putanje na **istom** ograničenju takta.

10.6. Poređenje sa referentnim rešenjem

Sistem se poredi sa rešenjem iz prethodne faze projekta, u kome je isto jezgro dobijeno visokonivoskom sintezom iz jezika C. Poređenje ima smisla po resursima i po broju taktova na jedinicu posla; ukupno vreme obrade cele table nije direktno uporedivo, o čemu na kraju odeljka.

Veličina	Referentno rešenje	Ovaj rad
LUT po instanci	5.269	1.910
DSP po instanci	16	9
Blok-memorija po instanci	4 manja bloka	19 većih ← slabije
LUT za dve instance	10.538 (59,9 %)	ceo sistem 6.261 (35,6 %)
Taktova za 90×90 / 30×30	10.360.183	3.783.652
Taktova po piksel-operaciji	3,09	1,13

Tabela 24. Poređenje sa rešenjem iz visokonivoske sinteze

Ceo naš sistem — dve instance, interkonekt i DMA zajedno — staje u manje LUT-ova nego što referentno rešenje troši samo na svoja dva jezgra. Po broju taktova na istom zadatku razlika je **2,74×** u našu korist. Jedina veličina po kojoj smo slabiji je blok-memorija, sa poznatim uzrokom (odjeljak 10.2).

Kako čitati brojku „taktova po piksel-operaciji“. To je ukupan broj taktova podeljen brojem množenja koje algoritam mora izvesti, i služi da se porede merenja izvedena na različitim veličinama zadatka. Vrednost **nije konstanta**: raste kako šablon opada, jer se režija od 112 taktova po poziciji prozora amortizuje na manje množenja. Na šablonu 30×30 iznosi 1,13, a na 25×15 čak 1,31. U tabeli je zato navedena vrednost na **istoj** veličini zadatka na kojoj je merena i referenca.

Zašto se ukupno vreme obrade table ne poredi tabelarno. Sistemski model prethodne faze kalibrisan je konstantom koja je izvedena iz izveštaja visokonivoske sinteze, dakle iz *tude* implementacije istog algoritma. Pošto naš RTL isti posao obavlja **2,74×** efikasnije po piksel-operaciji, ukupno vreme koje taj model predviđa nije uporedivo sa našim bez prethodne rekalkulacije te konstante. Poređenje po taktovima i po resursima, koje je u tabeli iznad, ne zavisi od te kalibracije i zato je navedeno.

11. Zaključak

RT model NCC jezgra je opisan u VHDL-u, funkcionalno verifikovan, sintetizovan i implementiran za ciljni uređaj xc7z010c1g400-1. Ključni rezultati:

- rezultat je **bit-identičan** referentnom modelu u jeziku C, na sintetičkom i na stvarnim podacima, i ostaje bit-identičan i kroz AXI omotač (odjeljak 7.5);
- samostalno jezgro **zatvara vremenska ograničenja** i pri 100 MHz, sa rezervom od 0,146 ns posle rutiranja i ostvarivom frekvencijom od približno 101,5 MHz (odjeljak 7.3);
- zauzeće resursa golog jezgra (8,36 % LUT, 11,25 % DSP, 15 % blok-memorije, **0** LUT-ova kao memorija) ostavlja dovoljno prostora za drugu instancu jezgra i za AXI infrastrukturu, što je preduslov za paralelizaciju predviđenu sistemskim modelom (odjeljak 7.2);
- latencija po pozivu iznosi 2.461.201 takt, odnosno 1,31 takta po piksel-operaciji; pri taktu integrisanog sistema od 90,909 MHz to je 27,07 ms i približno 185.300 pozicija prozora u sekundi (odjeljak 7.4).

Jezgro je zatim spakovano u IP blok sa dva AXI interfejsa i integrisano u sistem sa procesorskim sistemom Zynq, drugom instancom akceleratora i DMA kontrolerom. Rezultati na nivou celog sistema:

- izlaz kroz AXI omotač je **bit-identičan** golom jezgru, čime je verifikacija iz poglavlja 7 prenesena na spakovan blok bez ponovnog dokazivanja (odjeljak 7.5);
- ceo sistem zauzima **35,6 % LUT-ova** uređaja — **manje nego što referentno rešenje troši samo na svoja dva jezgra** (59,9 %) — i po instanci 1.910 naspram 5.269 LUT, odnosno 9 naspram 16 DSP blokova (odjeljci 10.2 i 10.6);
- sistem **zatvara vremenska ograničenja** na 90,909 MHz, sa rezervom od +0,170 ns i bez ijedne od 15.459 krajnjih tačaka koja krši ograničenje (odjeljak 10.3);
- na istom zadatku sistem troši **2,74× manje taktova** od referentnog rešenja: 3.783.652 naspram 10.360.183 (odjeljak 10.6).

Dve slabosti se navode otvoreno. Prva je blok-memorija: 65 % zauzeća, i to je jedini resurs bez rezerve, a po njemu smo slabiji od referentnog rešenja. Uzrok je 32-bitna reč integralne slike kod koje je dovoljno 21 bit; suženje bi oslobodilo oko šest blokova (odjeljak 10.2). Druga je radni takt: 90,909 MHz umesto nominalnih 100, dakle odstupanje od 9,1 %. Pri tome samostalno jezgro 100 MHz *zatvara* — ograničenje unosi integracija, kroz pritisak razmeštaja i zagušenje rutiranja, a ne RTL (odjeljak 10.3).

Preostaje generisanje konfiguracione datoteke i provera na realnoj ploči, uz softver prenesen iz izvršne specifikacije. Interfejs koji taj softver treba da koristi — adresna mapa, registri, tok podataka i dva obavezna pravila — opisan je u poglavljima 8 i 9.